

ORBITUNE - 3D Real-like Music Experience

A MAJOR PROJECT REPORT

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF
THE DEGREE OF

BACHELOR OF COMPUTER APPLICATIONS

(specialization in Data Science)

SUBMITTED BY

Yuvraj Singh Kushwah(2023BCAH062)

Subhro Pal (2023BCAH056)

UNDER SUPERVISION OF

Ms. Shivangee Nagar

Assistant Professor

December-2025



JAGRAN SCHOOL OF COMPUTER APPLICATIONS (JSCA)

FACULTY OF SCIENCE AND TECHNOLOGY (FAST)

JAGRAN LAKECITY UNIVERSITY, BHOPAL (M.P.)

ACKNOWLEDGMENT

It is indeed a great pleasure to express my/our thanks and gratitude to all those who helped me/us during this period. This project would not have been materialized without the help from many quarters. We, sincerely thank to all the persons who ever played a vital role in the successful completion of our project.

We sincerely thank all the people who co-operate and encourage me throughout the semester and make my project work successful.

We are thankful to **Ms. Shivangee Nagar** (Project Mentor, JSCA, FAST) who has constantly remained helpful in suggesting directions and providing me guidance throughout the project.

We also thank **Ms. Vaishali Sharma** (Program Leader (BCA), JSCA) for providing us the confidence and helping us in the project.

Special thanks to **Dr. Dileep Kumar Singh** (Head, JSCA) for providing this platform which helped in the development of our technical skills and provided opportunity to study and work here which imparted a lot of knowledge, experience, confidence, skills, and real field awareness to us.

It is good fortune that we had support and well wishes of many. We thank all those, whose names have not appeared here but the contributions have not gone unnoticed.

Name: Yuvraj Singh Kushwah

Signature:

Roll No: 2023BCAH062

Name: Subhro Pal

Signature:

Roll No: 2023BCAH056

CERTIFICATE

We hereby certify that the work which is being presented in the BCA Hons. Major Project Report entitled “**ORBITUNE - 3D Real-like Music Experience**”, in the partial fulfillment of the requirements for the award of the **Bachelor of Computer Applications (Data Science)** is an authentic record of our own work carried out during session **Jul-Dec, 2025 (5th semester)** under the guidance of **Ms. Shivangee Nagar, Assistant Professor, JSCA.**

The matter presented in this Project Report has not been submitted by us for the award of any other degree/diploma elsewhere.

Name : Yuvraj Singh Kushwaha
Roll No: 2023BCAH062

Signature:

Name : Subhro Pal
Roll No: 2023BCAH056

Signature:

This is to certify that the above statement made by the students is correct to the best of my knowledge.

Date:

Miss. Shivangee Nagar
Project Guide, Assistant Professor,
JSCA, JLU Bhopal

Mr. Priyank Dubey
Project Coordinator (BCA)
JSCA, JLU Bhopal

Dr. Dileep Kumar Singh
Head
JSCA, JLU Bhopal

DECLARATION

I hereby declare that the Major Project Report entitled “ORBITUNE - 3D Real-like Music Experience” submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Computer Applications (BCA) to Jagran Lakecity University, is the result of my own work carried out under the guidance of Ms. Shivangee Nagar, Assistant Professor, JSCA.

I further declare that this project report has not been previously submitted by me or any other person for the award of any degree or diploma of this or any other university or institution, and that all the sources of information used in the project have been duly acknowledged.

Place:

Date:

Signature of the Student
Yuvraj Singh Kushwah
Roll No.: 2023BCAH062

Signature of the Student
Subhro Pal
Roll no. – 2023BCAH056

Certified by:

Ms. Shivangee Nagar
Assistant Professor
JSCA

Table of Contents

ACKNOWLEDGMENT	i
CERTIFICATE.....	ii
DECLARATION	iii
ABSTRACT.....	ix
CHAPTER 1 - INTRODUCTION.....	1
1.1 Background and Motivation	1
1.2 Problem Statement.....	2
1.3 Objectives and Scope.....	4
1.4 Significance of the Study	7
CHAPTER 2 - LITERATURE REVIEW.....	9
2.1 Introduction.....	9
2.2 Spatial Audio Technologies	9
2.2.1 Fundamentals of Human Sound Localization	9
2.2.2 Head-Related Transfer Functions	10
2.2.3 Binaural Audio Rendering	10
2.2.4 Room Acoustics and Reverberation.....	11
2.3 Audio Source Separation	11
2.3.1 Traditional Approaches.....	11
2.3.2 Deep Learning Revolution	12
2.3.3 Practical Considerations.....	12
2.4 Conversational AI and Natural Language Processing	13
2.4.1 Evolution of Language Models.....	13
2.4.2 Intent Detection and Classification.....	13
2.4.3 Context and Memory in Conversations	14
2.5 Music Information Retrieval and Recommendation	14
2.5.1 Content-Based Music Analysis.....	14
2.5.2 Collaborative Filtering and Recommendation	15
2.5.3 Genre Classification and Music Understanding.....	15
2.6 Web-Based Audio Processing Systems	16
2.6.1 Browser-Based Audio Technologies	16
2.6.2 Client-Server Architecture for Audio Processing	16
2.7 Gap Analysis and Research Positioning	16

CHAPTER 3 - SYSTEM DESIGN	18
3.1 Introduction.....	18
3.2 System Architecture Overview	18
3.2.1 High-Level Architecture	18
3.2.2 Design Principles	19
3.3 Frontend Architecture	20
3.3.1 Technology Stack and Framework Selection.....	20
3.3.2 State Management Architecture.....	21
3.3.3 Component Architecture	21
3.3.4 API Communication Layer	22
3.4 Backend Architecture.....	22
3.4.1 Framework and API Design.....	22
3.4.2 Request Processing Flow	23
3.4.3 Data Persistence Strategy.....	24
3.4.4 Static File Serving.....	24
3.5 AI/ML Processing Architecture	25
3.5.1 Processing Pipeline Design.....	25
3.5.2 Genre Detection Subsystem	26
3.5.3 HRTF Processing Implementation.....	26
3.5.4 GPU Acceleration Strategy	27
3.6 Conversational AI Architecture	27
3.6.1 Multi-Stage Intent Detection.....	27
3.6.2 Response Generation	28
3.6.3 User Profiling and Memory	28
3.7 Data Flow Architecture.....	29
3.7.1 Request-Response Patterns	29
3.7.2 File Storage Organization	29
3.7.3 Error Propagation and Handling	30
3.8 Scalability Considerations	30
3.8.1 Current Limitations	30
3.8.2 Future Scaling Paths	30
CHAPTER 4 - METHODOLOGY.....	32
4.1 Introduction.....	32

4.2 Audio Acquisition Methodology	32
4.2.1 YouTube Content Extraction	32
4.2.2 Metadata Extraction and Management	33
4.3 Source Separation Methodology	33
4.3.1 Model Selection and Configuration	33
4.3.2 Processing Pipeline	34
4.3.3 Quality Assurance	35
4.4 Spatial Audio Processing Methodology	35
4.4.1 HRTF Implementation Approach	35
4.4.2 Carousel Rotation System	36
4.4.3 Distance Modeling	37
4.4.4 Dynamic Distance Variation	37
4.5 Audio Enhancement Methodology	38
4.5.1 Stem-Specific Equalization	38
4.5.2 Reverberation Processing	39
4.5.3 Mastering Chain	40
4.6 Conversational AI Methodology	41
4.6.1 Intent Classification Strategy	41
4.6.2 Response Generation Approach	42
4.6.3 User Profiling Methodology	43
4.7 User Interface Design Methodology	44
4.7.1 Interaction Mode Switching	44
4.7.2 Progressive Disclosure Strategy	45
4.7.3 Feedback and Progress Communication	45
4.8 Optimization Methodology	46
4.8.1 GPU Acceleration Strategy	46
4.8.2 Caching and Reuse	46
CHAPTER 5 - IMPLEMENTATION	47
5.1 Introduction	47
5.2 Technology Stack	47
5.2.1 Backend Technologies	47
5.2.2 Frontend Technologies	47
5.3 Backend Implementation	48

5.3.1 API Structure	48
5.3.2 Data Persistence	48
5.3.3 Processing Orchestration.....	49
5.4 Audio Processing Implementation.....	49
5.4.1 YouTube Integration.....	49
5.4.2 Source Separation	49
5.4.3 Spatial Processing.....	49
5.4.4 Genre Detection	50
5.5 Conversational AI Implementation.....	50
5.5.1 Intent Detection.....	50
5.5.2 Response Generation	50
5.5.3 User Profiling.....	51
5.6 Frontend Implementation.....	51
5.6.1 Component Architecture	51
5.6.2 State Management.....	51
5.6.3 API Communication	51
5.6.4 Audio Visualization	52
5.7 Deployment Strategy	52
5.7.1 Dependency Management.....	52
5.7.2 Startup Automation.....	52
5.8 Challenges and Solutions.....	53
5.8.1 YouTube Reliability.....	53
5.8.2 Memory Constraints.....	53
5.8.3 Audio Quality.....	53
5.8.4 AI Consistency.....	53
CHAPTER 6 - RESULTS AND ANALYSIS.....	54
6.1 Introduction.....	54
6.2 System Performance Analysis	54
6.2.1 Processing Time Measurements.....	54
6.2.2 Memory and Storage Requirements.....	54
6.3 Audio Quality Assessment.....	55
6.3.1 Source Separation Quality	55
6.3.2 Spatial Audio Quality	55

6.4 Conversational AI Performance.....	56
6.4.1 Intent Detection Results.....	56
6.4.2 Response Quality Assessment	56
6.5 User Interface and Experience	57
6.5.1 Interface Overview.....	57
6.5.2 Usability Testing Results	58
6.5.3 User Feedback.....	59
6.6 System Reliability.....	59
6.6.1 Success Rates	59
6.6.2 Stability and Cross-Platform Testing.....	60
6.7 Comparative Analysis.....	60
6.7.1 Comparison with Existing Solutions.....	60
6.7.2 Performance Against Project Objectives	61
6.8 Key Findings Summary	61
CHAPTER 7 - CONCLUSION.....	63
REFERENCES	

ABSTRACT

ORBITUNE is a web-based platform designed to transform any standard audio track into a breathtaking 3D spatial audio experience. Our goal is to make immersive listening accessible to everyone, turning everyday music from sources like YouTube into a rich, multi-dimensional soundscape that feels like you're right there in the room with the artists.

At its heart, ORBITUNE uses a sophisticated AI-powered pipeline. When a user provides a song, our system intelligently deconstructs the track into its fundamental components—vocals, bass, drums, and other instruments. Then, like a sound engineer in a virtual studio, it strategically places each of these elements in a three-dimensional space. This process simulates a realistic acoustic environment, creating a listening experience that is both expansive and deeply engaging. Ultimately, ORBITUNE represents an exploration into the future of digital audio. By merging advanced AI for source separation, spatial audio algorithms, and interactive web design, this project demonstrates a powerful new way for us to connect with the music we love. It's a step towards breaking the flat plane of stereo and inviting listeners to truly immerse themselves in the art of sound

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

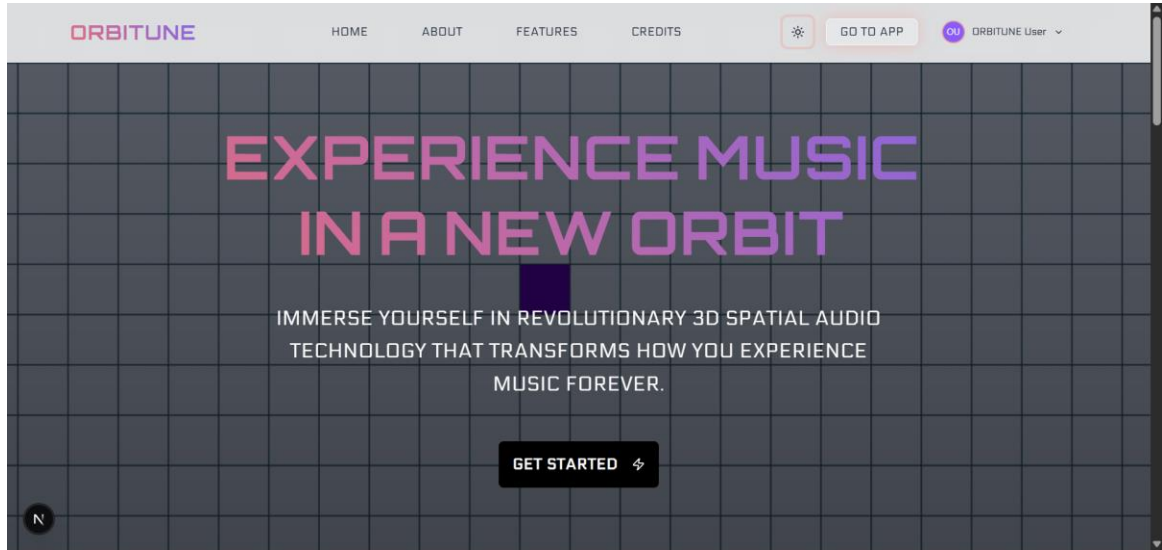


Figure 1.1 – Homepage of the project (introductory page)

The evolution of music consumption has undergone remarkable transformation over the past few decades. What began with physical media like vinyl records and compact discs has now shifted almost entirely to digital streaming platforms, fundamentally changing how people discover and experience music. However, despite these technological advances, the majority of music content remains confined to traditional stereo playback, which represents a relatively flat and limited listening experience compared to how humans naturally perceive sound in three-dimensional space [1].

In real-world environments, humans possess the remarkable ability to localize sound sources with precision, determining not only the direction from which sounds originate but also their distance and movement patterns. This spatial awareness is crucial for our everyday interactions and has been honed through millions of years of evolution [2]. When listening to music through conventional stereo systems or headphones, much of this spatial information is either absent or poorly represented, resulting in a listening experience that feels confined and unnatural.

The concept of spatial audio is not entirely new. Professional music production has long employed surround sound systems in studios, and technologies like Dolby Atmos have brought some degree of spatial audio to cinema and high-end home theater setups [3]. However, these solutions typically require specialized hardware, proprietary content formats, and technical expertise that place them beyond the reach of average music listeners. The disconnect between the potential of spatial audio and its practical accessibility represents a significant gap in current audio technology.

Recent breakthroughs in artificial intelligence have opened new possibilities for audio processing that were previously unimaginable. Deep learning models can now analyze complex audio signals and separate them into individual instrument components with accuracy that rivals human experts [4]. This capability, known as source separation, forms the foundation for many modern audio applications, from karaoke systems to music production tools. Similarly, advances in natural language processing have enabled conversational interfaces that can understand user intent with remarkable sophistication [5].

The platform is built on the observation that most people discover music through YouTube, which hosts an enormous catalog covering virtually every musical style and era. Rather than requiring users to purchase specific tracks or subscribe to premium services, ORBITUNE works directly with YouTube content, transforming any video into an immersive spatial audio experience. This approach significantly lowers the barrier to entry while maximizing content availability.

1.2 Problem Statement

The current landscape of music streaming and audio technology faces several interconnected challenges that ORBITUNE aims to address:

Limited Accessibility to Spatial Audio: While spatial audio technologies exist, they remain largely inaccessible to the general public. Professional audio production tools require significant investment in both software licenses and acoustic expertise. Consumer-facing solutions like Apple Spatial Audio or Sony 360 Reality Audio are limited to specific hardware ecosystems and curated content libraries [7]. The average

music listener has no practical way to experience their favorite songs in true 3D audio without these constraints.

Content Format Limitations: Music streaming platforms primarily distribute content in stereo format, which has been the standard for decades. Although some services have begun offering spatial audio tracks, the conversion process requires manual remixing by audio engineers, making it economically viable only for popular releases. This creates a situation where spatial audio remains a premium feature rather than a standard listening option, leaving the vast majority of musical content—including older recordings, niche genres, and regional music—unavailable in spatial formats [8].

Technical Complexity Barriers: Existing audio processing tools present steep learning curves that intimidate casual users. Professional digital audio workstations feature hundreds of parameters, requiring users to understand technical concepts like equalization, compression ratios, and frequency response curves. Even simplified consumer applications often assume a level of audio engineering knowledge that most people lack. This complexity creates a barrier between users who want enhanced audio experiences and the tools that could provide them.

Quality Inconsistency in Amateur Solutions: The desire for spatial audio has led to numerous amateur conversions available on YouTube and other platforms, often labeled as "8D audio" or "3D audio." However, these conversions frequently suffer from severe quality issues. Many employ simplistic panning techniques that merely move audio back and forth between left and right channels, creating an artificial and fatiguing listening experience rather than genuine spatial positioning [9]. Others apply excessive reverberation or processing artifacts that degrade audio quality. The lack of standardization and quality control in these amateur efforts has, in some cases, given spatial audio itself a poor reputation.

Disconnect Between User Intent and Technical Implementation: Traditional music interfaces require users to know exactly what they want to hear and express it in explicit search terms. This approach fails to accommodate the reality of how people actually think about music selection, which is often based on emotional states, activities, or abstract

preferences. A user might feel "melancholic but hopeful" or want "something to help me focus while working," but translating these feelings into specific artist names or song titles requires cognitive effort that disrupts the natural flow of music discovery.

Lack of Personalization in Spatial Processing: Different musical genres have different spatial characteristics that suit them best. A classical orchestra recording benefits from a spacious concert hall ambiance, while electronic dance music often works better with tight, focused positioning. Current spatial audio solutions typically apply uniform processing regardless of content, missing opportunities to tailor the spatial presentation to match the musical style and enhance the listening experience [10].

Computational Resource Requirements: High-quality audio processing, particularly tasks like source separation and binaural rendering, demands significant computational power. Most attempts to democratize these technologies have been hampered by the need for expensive hardware or long processing times that make real-world usage impractical. Without efficient implementation, spatial audio processing risks remaining a laboratory curiosity rather than a practical consumer technology.

1.3 Objectives and Scope

The primary objective of ORBITUNE is to develop an integrated platform that makes professional-quality 3D spatial audio accessible to everyone while providing an intelligent, natural interface for music discovery and interaction. This overarching goal breaks down into several specific objectives:

Primary Objectives:

1. Automated Audio Acquisition and Processing Pipeline

- Implement a robust system for downloading high-quality audio from YouTube while handling various video formats and quality levels
- Develop error handling mechanisms for network issues, unavailable content, and format changes
- Ensure metadata preservation including song titles, artists, and thumbnail images for proper content organization

2. AI-Powered Source Separation

- Integrate state-of-the-art deep learning models for separating mixed audio into individual instrument stems
- Optimize the separation process for both quality and speed, balancing computational requirements with output fidelity
- Handle diverse musical genres and recording qualities, from studio productions to live performances

3. Professional-Grade Spatial Audio Engine

- Implement Head-Related Transfer Function processing for realistic binaural spatialization
- Develop a carousel-based rotation system that positions different instruments around the listener in three-dimensional space
- Apply distance modeling and room acoustics simulation to enhance realism
- Create genre-specific processing profiles that adapt spatial characteristics to match musical content

4. Intelligent Conversational Interface

- Build an AI-powered chatbot using modern language models to understand both explicit music queries and implicit mood-based requests
- Implement multi-stage intent detection to differentiate between direct search queries and conversational interactions
- Develop user profiling capabilities that learn preferences over time and provide increasingly personalized recommendations

5. Seamless User Experience

- Design an intuitive interface that requires no technical knowledge or audio engineering expertise
- Provide real-time feedback during processing stages to keep users informed

- Enable smooth transitions between different interaction modes (search versus conversation)
- Implement audio visualization features that make the spatial positioning visually apparent

6. Performance Optimization

- Leverage GPU acceleration where available to dramatically reduce processing times
- Implement caching strategies to avoid redundant computation for previously processed content
- Optimize memory usage to enable processing on consumer-grade hardware

Scope and Limitations:

The scope of ORBITUNE encompasses several key areas while acknowledging certain practical constraints:

In Scope:

- Processing of publicly available YouTube audio content up to 10 minutes in duration
- Support for common music genres including pop, rock, electronic, classical, jazz, and hip-hop
- Real-time conversational interaction with response times under one second
- Playback on standard stereo headphones or speakers (binaural audio)
- User preference learning and recommendation based on listening history
- Cross-platform compatibility through web-based interface

Out of Scope:

- Live concert streaming or real-time audio processing during recording
- Support for proprietary audio formats requiring specific licensing
- Multi-channel surround sound output requiring specialized speaker setups

- Commercial distribution or licensing of processed audio content
- Mobile native applications (initial release focuses on web platform)
- Processing of audio content longer than 10 minutes due to computational constraints

Technical Constraints:

- Processing time varies with audio length and hardware capabilities, typically ranging from 2-5 minutes per song on GPU-enabled systems
- Audio quality depends on source material quality from YouTube
- Spatial audio effects are optimized for headphone listening; speaker playback provides reduced spatial perception
- Internet connection required for YouTube content access and AI model interactions

1.4 Significance of the Study

The development of ORBITUNE carries significance across multiple dimensions, from technological innovation to potential social impact:

Technological Advancement: This project demonstrates the practical integration of several cutting-edge AI technologies into a cohesive system. While source separation models, spatial audio algorithms, and conversational AI have been studied independently, their combination into a unified platform represents a novel contribution. The work particularly advances the practical application of these technologies beyond laboratory settings into real-world consumer contexts [11].

Democratization of Audio Technology: By removing technical and economic barriers to spatial audio experiences, ORBITUNE contributes to the broader goal of democratizing advanced audio technologies. This aligns with ongoing trends in technology development where sophisticated capabilities gradually become accessible to wider audiences. The platform enables anyone with basic computing resources to experience professional-grade audio processing that would otherwise require expensive equipment and specialized knowledge.

Educational Value: The project serves as a comprehensive case study in full-stack development, encompassing frontend design, backend architecture, machine learning integration, and audio signal processing. It demonstrates practical solutions to real-world challenges like optimizing deep learning inference, managing stateful conversational interactions, and implementing responsive user interfaces for computationally intensive tasks.

Human-Computer Interaction Innovation: The conversational interface represents an evolution in how users interact with music systems. Rather than forcing users to adapt to rigid search paradigms, the system adapts to natural human communication patterns. This shift toward more intuitive interaction models has broader implications for software design across domains [12].

Content Accessibility: By working with YouTube's vast content library, ORBITUNE makes spatial audio versions of rare recordings, regional music, and older content available in formats they were never originally released in. This has potential cultural significance in preserving and enhancing access to diverse musical heritage.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter examines the existing body of knowledge that forms the theoretical and technical foundation for ORBITUNE. The review is organized into five major areas: spatial audio technologies and psychoacoustics, audio source separation techniques, conversational AI and natural language processing, music recommendation systems, and web-based audio processing platforms. Each section critically analyzes relevant research, identifies gaps in current solutions, and establishes the context for the design decisions made in this project.

2.2 Spatial Audio Technologies

2.2.1 Fundamentals of Human Sound Localization

Human spatial hearing relies on several psychoacoustic cues that the auditory system uses to determine the location of sound sources in three-dimensional space. The two primary binaural cues are Interaural Time Difference (ITD) and Interaural Level Difference (ILD). ITD refers to the difference in arrival time of a sound wave at each ear, which varies based on the azimuthal angle of the source [1]. For a sound source directly to the right, the sound reaches the right ear approximately 0.6 milliseconds before the left ear. This time difference, though seemingly minute, provides crucial information for horizontal localization.

ILD, also known as Interaural Intensity Difference, describes the difference in sound pressure level between the two ears. This occurs because the head acts as an acoustic obstacle, creating a "shadow" effect for high-frequency sounds [2]. The magnitude of this shadow depends on both the frequency of the sound and the angle of incidence. High frequencies above approximately 1500 Hz are particularly affected, while lower frequencies diffract around the head more easily, making ILD less significant for bass-range sounds.

Beyond these primary cues, spectral cues provided by the shape of the outer ear (pinna) play a crucial role in elevation perception and front-back disambiguation. The complex

folds and ridges of the pinna create direction-dependent filtering that modifies the frequency spectrum of incoming sounds in characteristic ways [1]. This filtering is highly individual, which explains why spatial audio experiences can vary significantly between listeners when using generic processing.

2.2.2 Head-Related Transfer Functions

Head-Related Transfer Functions represent the mathematical description of how sounds are filtered by the head, torso, and pinnae as they travel from a source location to the eardrums. These functions capture all the acoustic transformations that occur, including ITD, ILD, and spectral modifications [2]. HRTF data is typically measured in anechoic chambers by placing microphones in the ear canals of subjects and recording impulse responses from loudspeakers positioned at various angles and distances.

The challenge with HRTFs lies in their individualized nature. While generic HRTF datasets like the CIPIC database provide measurements from multiple subjects that can serve as reasonable approximations for general use, truly accurate spatial audio requires personalized measurements [10]. Some research has explored using anthropometric measurements to select appropriate HRTFs from databases or even synthesizing individualized HRTFs, but these approaches remain complex and not widely deployed in consumer applications.

In practice, most spatial audio systems use generic HRTFs carefully selected to provide acceptable results for the majority of listeners. The trade-off between personalization accuracy and practical implementation represents an ongoing challenge in the field.

2.2.3 Binaural Audio Rendering

Binaural audio rendering involves convolving monophonic or multi-channel audio signals with appropriate HRTFs to create the illusion of sound sources positioned in three-dimensional space when played through headphones. The process requires careful consideration of several factors beyond simply applying HRTF filters. Distance perception, for instance, relies on amplitude decay, the direct-to-reverberant energy ratio, and high-frequency air absorption effects [3].

Dynamic binaural rendering adds another layer of complexity by tracking head movements and adjusting the spatial audio in real-time to maintain stable source positions relative to the environment rather than the listener's head. This significantly enhances immersion and reduces the common front-back confusion that can occur with static binaural audio. However, dynamic rendering requires head-tracking sensors and additional computational resources, limiting its application primarily to virtual reality contexts.

2.2.4 Room Acoustics and Reverberation

Natural listening environments contain reflections from walls, floors, and other surfaces that contribute significantly to spatial perception. Early reflections arriving within the first 50 milliseconds after the direct sound provide spatial information about room size and shape, while later reverberation creates a sense of envelopment and ambiance [10]. Artificial reverberation algorithms attempt to recreate these characteristics, with approaches ranging from simple algorithmic reverbs to complex convolution-based methods using measured impulse responses from real spaces.

The perception of room acoustics varies considerably with musical genre and production style. Classical music recordings typically benefit from spacious hall ambiance that recreates the concert hall experience, while popular music often employs tighter, more controlled reverberation to maintain clarity and punch. This genre-dependent relationship between spatial processing and aesthetic quality informed the decision to implement adaptive processing profiles in ORBITUNE.

2.3 Audio Source Separation

2.3.1 Traditional Approaches

Audio source separation—the task of isolating individual instruments or sound sources from mixed audio recordings—has been a longstanding challenge in signal processing. Early approaches relied on exploiting differences in frequency content, spatial positioning (in multi-channel recordings), or temporal patterns. Independent Component Analysis (ICA) and Non-negative Matrix Factorization (NMF) represented classical methods that

showed promise but ultimately struggled with the complexity of real-world music mixtures [4].

These traditional methods operated under assumptions that often failed to hold in practice. For instance, ICA assumes statistical independence between sources, which is frequently violated in music where instruments are intentionally mixed to blend harmoniously. NMF assumes that signals can be represented as additive combinations of non-negative basis functions, but this mathematical constraint doesn't necessarily align with the physical reality of how musical sounds combine.

2.3.2 Deep Learning Revolution

The application of deep neural networks to source separation marked a paradigm shift in capability. Convolutional neural networks and recurrent architectures demonstrated unprecedented performance by learning complex patterns from large datasets of mixed and separated audio [11]. These models could implicitly learn the characteristic timbres, temporal structures, and harmonic relationships of different instruments without requiring hand-crafted features or explicit assumptions about signal structure.

The development of the Demucs model represented a significant milestone in this evolution. Unlike earlier approaches that operated in the frequency domain using spectrograms, Demucs processes audio directly in the time domain (waveform), avoiding artifacts associated with time-frequency transformations [4]. The hybrid transformer version (htdemucs) combines convolutional layers for local pattern recognition with transformer attention mechanisms for capturing long-range dependencies, achieving state-of-the-art separation quality across diverse musical styles.

2.3.3 Practical Considerations

Despite impressive performance metrics, deploying source separation models in real-world applications involves numerous practical challenges. Computational requirements are substantial—processing a three-minute song can take several minutes on CPU-only systems. The models also require significant memory, particularly when processing high sample rates necessary for audio fidelity. Furthermore, separation quality varies with the

complexity of the input material; dense, heavily processed mixes prove more challenging than sparse acoustic recordings.

The choice of how many sources to separate represents another design consideration. While separating into just two stems (vocals and accompaniment) is computationally simpler, finer-grained separation into individual instruments enables more sophisticated spatial positioning. The four-stem approach (vocals, drums, bass, other) adopted by Demucs and used in ORBITUNE strikes a practical balance between separation granularity and computational feasibility [4].

2.4 Conversational AI and Natural Language Processing

2.4.1 Evolution of Language Models

Natural language processing has undergone dramatic transformation with the development of large language models based on transformer architecture. These models, trained on vast text corpora, demonstrate remarkable ability to understand context, generate coherent responses, and perform complex reasoning tasks [5]. The progression from rule-based chatbots to statistical models and finally to neural language models represents a fundamental shift in how computers process and generate human language.

The introduction of models like GPT (Generative Pre-trained Transformer) and its successors demonstrated that language models could be "few-shot learners," adapting to new tasks with minimal task-specific training [5]. This capability makes them particularly well-suited for conversational applications where user queries can vary unpredictably and may not fit neatly into predefined categories.

2.4.2 Intent Detection and Classification

Understanding user intent—determining what a user wants to accomplish from their natural language input—forms the foundation of effective conversational systems. Traditional intent detection relied on pattern matching and keyword extraction, which proved brittle when faced with the natural variation in how people express themselves. Modern approaches using neural networks can learn more robust representations that generalize better to unseen phrasings [12].

Multi-stage intent detection, where initial fast heuristics handle common cases and more sophisticated models engage only when needed, offers a practical compromise between latency and accuracy. This hybrid approach minimizes response time for straightforward queries while maintaining capability to handle complex or ambiguous inputs that require deeper analysis.

2.4.3 Context and Memory in Conversations

Effective conversational systems maintain context across multiple turns, allowing them to interpret pronouns, track evolving topics, and build on previous exchanges. Short-term conversational memory captures the immediate dialogue history, while long-term memory preserves user preferences and historical interactions [12]. The challenge lies in determining what information to retain, how to represent it efficiently, and when to retrieve it for influencing current responses.

User modeling extends beyond simple preference tracking to infer deeper patterns about musical taste, emotional associations, and contextual factors that influence music selection. These models can identify that a user prefers upbeat music in mornings but mellower sounds in evenings, or that certain artists consistently receive positive feedback regardless of mood.

2.5 Music Information Retrieval and Recommendation

2.5.1 Content-Based Music Analysis

Music Information Retrieval (MIR) encompasses techniques for extracting meaningful information from audio signals. Feature extraction methods identify characteristics like tempo, key, timbre, and rhythmic patterns that can be used for similarity measurement, genre classification, and mood detection [8]. Traditional MIR relied heavily on hand-crafted features designed by audio engineers and music theorists, such as Mel-frequency cepstral coefficients for timbre description or onset detection functions for rhythm analysis.

Deep learning has also transformed this domain, with convolutional neural networks learning to extract relevant features directly from audio spectrograms or waveforms.

These learned representations often capture musical characteristics that weren't explicitly designed for, sometimes outperforming hand-crafted features on downstream tasks.

2.5.2 Collaborative Filtering and Recommendation

Music recommendation systems typically combine multiple approaches. Collaborative filtering identifies patterns across user listening histories to suggest music that similar users enjoyed. Content-based filtering recommends music with similar acoustic characteristics to what a user has liked previously. Hybrid systems combine both approaches to leverage their complementary strengths [8].

The cold start problem—how to make recommendations for new users with limited history—remains challenging. Context-aware recommendation that considers time of day, current activity, or explicitly stated mood can provide useful signals even without extensive listening history. This contextual approach aligns with how ORBITUNE's conversational interface operates, using immediate conversational context to inform recommendations.

2.5.3 Genre Classification and Music Understanding

Automatic genre classification attempts to categorize music into stylistic categories, though genre boundaries are often fuzzy and subjective. Traditional classification approaches used combinations of acoustic features with machine learning classifiers, while modern methods employ deep neural networks trained on large labeled datasets [8]. However, genre labels represent simplifications of complex musical characteristics, and relying solely on genre can lead to oversimplified recommendations.

More nuanced music understanding considers multiple dimensions like valence (emotional positivity/negativity), energy level, danceability, and acoustic versus electronic characteristics. This multi-dimensional representation provides richer information for matching music to listener preferences and contextual requirements.

2.6 Web-Based Audio Processing Systems

2.6.1 Browser-Based Audio Technologies

Modern web browsers support sophisticated audio processing through the Web Audio API, which provides low-level audio manipulation capabilities including real-time synthesis, effects processing, and spatialization [7]. This enables complex audio applications to run directly in browsers without plugins or native applications. However, computationally intensive tasks like source separation remain impractical to run client-side, necessitating server-side processing architectures.

Progressive Web Apps and modern JavaScript frameworks enable responsive, application-like experiences that blur the line between web pages and native applications. This architectural approach provides broad accessibility while maintaining sophisticated functionality, though it requires careful attention to state management, loading strategies, and network communication patterns.

2.6.2 Client-Server Architecture for Audio Processing

Processing-intensive audio applications typically adopt architectures where heavy computation occurs on servers with powerful hardware (particularly GPUs) while clients handle user interaction and audio playback [9]. This distributed approach must address challenges like managing processing queues, providing progress feedback during long operations, and efficiently transferring processed audio files.

Caching strategies become crucial for performance—storing processed results allows instant retrieval if the same content is requested again, avoiding redundant computation. However, storage costs and cache invalidation policies must be carefully managed as processed audio files can be quite large.

2.7 Gap Analysis and Research Positioning

While extensive research exists in each domain covered above, several gaps remain that ORBITUNE addresses:

Integration Gap: Existing solutions typically focus on individual capabilities—either source separation tools, or spatial audio processors, or music recommendation systems—

but rarely integrate these into cohesive platforms. The few integrated solutions that exist operate primarily in professional production contexts with steep learning curves.

Accessibility Gap: Advanced audio processing technologies remain largely confined to professional environments or require specialized hardware. Consumer-facing spatial audio solutions are limited to curated content libraries rather than enabling processing of arbitrary audio [3].

Interface Gap: Most audio processing tools assume technical knowledge and present interfaces designed for audio engineers rather than casual listeners. Conversational interfaces for music systems typically focus on search and playback control but don't extend to controlling audio processing parameters [12].

Personalization Gap: While recommendation systems consider user preferences for content selection, spatial audio processing typically applies uniform processing regardless of musical genre or user preferences. The relationship between musical characteristics and optimal spatial presentation remains underexplored [10].

ORBITUNE positions itself to address these gaps by creating an integrated platform that combines state-of-the-art audio processing with intuitive conversational interaction, making professional-quality spatial audio accessible to general audiences while adapting processing to musical content characteristics.

CHAPTER 3

SYSTEM DESIGN

3.1 Introduction

This chapter presents the architectural design of ORBITUNE, describing how various components interact to deliver an integrated 3D spatial audio experience with conversational AI capabilities. The system design follows modern software engineering principles including separation of concerns, modularity, and scalability. The architecture is divided into three major tiers: the frontend presentation layer, the backend application layer, and the AI/ML processing layer, each with distinct responsibilities and well-defined interfaces.

3.2 System Architecture Overview

3.2.1 High-Level Architecture

ORBITUNE adopts a three-tier client-server architecture that separates user interface concerns from business logic and computationally intensive audio processing. This separation allows each layer to be developed, tested, and scaled independently while maintaining clean interfaces between components.

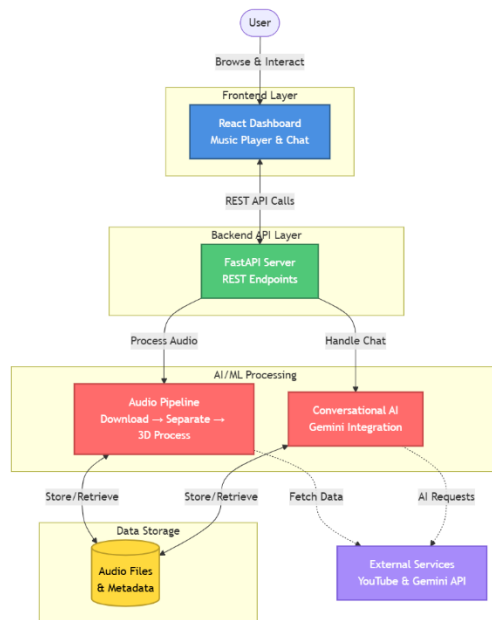


Figure 3.1 System Architecture diagram of the project

The **Frontend Layer** runs in the user's web browser and handles all user interactions, including the conversational interface, music player controls, audio visualization, and real-time feedback during processing. It communicates with the backend exclusively through RESTful API calls, maintaining stateless interactions that simplify scaling and error recovery.

The **Backend Layer** serves as the application coordinator, managing API endpoints, request routing, data persistence, and orchestration of the audio processing pipeline. Built on FastAPI, it provides high-performance asynchronous request handling and automatic API documentation. This layer also hosts the conversational AI service, which processes chat messages and generates appropriate responses.

The **AI/ML Processing Layer** contains the computationally intensive components responsible for audio downloading, source separation, and 3D spatial processing. These modules operate as callable services invoked by the backend layer, with processing occurring either on GPU hardware when available or falling back to CPU execution. This layer is designed to be resource-intensive but stateless, allowing for potential horizontal scaling across multiple processing nodes in future iterations.

3.2.2 Design Principles

Several core principles guided the architectural decisions:

Modularity: Each major function exists as a self-contained module with clear inputs and outputs. The source separator doesn't need knowledge of spatial processing, and the chatbot operates independently of audio processing logic. This modularity facilitates testing, maintenance, and future enhancements [11].

Graceful Degradation: The system adapts to available resources. Without GPU hardware, processing continues on CPU with longer wait times but identical output quality. If the conversational AI service encounters errors, users can still perform direct song searches. This resilience prevents single points of failure from breaking the entire system.

Progressive Enhancement: The interface provides immediate feedback and partial functionality while processing occurs in the background. Users see their query acknowledged instantly, receive progress updates during processing, and can continue exploring the interface while audio generation completes.

Separation of Concerns: User interface code knows nothing about HRTF processing algorithms, and audio processing modules don't handle HTTP requests. This separation simplifies reasoning about the system and allows specialists to focus on specific layers without needing to understand the entire codebase [11].

3.3 Frontend Architecture

3.3.1 Technology Stack and Framework Selection

The frontend is built using React 19, a mature and widely-adopted JavaScript library for building user interfaces. React's component-based architecture aligns well with the modular design philosophy, allowing the interface to be decomposed into reusable pieces like the music player, chat interface, and visualization components. The choice of React over alternatives like Vue or Angular was driven by its extensive ecosystem, strong community support, and excellent performance characteristics [6].

Next.js 15 provides the application framework, adding server-side rendering capabilities, automatic code splitting, and optimized production builds. While ORBITUNE currently operates primarily as a client-rendered application, Next.js enables potential future enhancements like server-side rendering for improved initial load times and better search engine optimization.

For styling, Tailwind CSS offers a utility-first approach that accelerates development while maintaining design consistency. Rather than writing custom CSS for every component, developers compose interfaces using pre-defined utility classes, reducing the overall stylesheet size and improving maintainability. Radix UI provides accessible, unstyled component primitives that ensure the interface works properly with keyboard navigation and screen readers, though accessibility represents an ongoing area for improvement.

3.3.2 State Management Architecture

Managing application state in complex interactive applications requires careful consideration. ORBITUNE employs React's Context API for global state management, avoiding the complexity of external libraries like Redux while providing sufficient capability for the application's needs. The MusicContext maintains the current playback state, queue, user preferences, and playlist data, making this information accessible throughout the component tree without prop drilling.

State persistence uses browser localStorage to preserve user preferences and playback history across sessions. When the application loads, it attempts to restore the previous state, providing continuity in the user experience. However, large data structures like the complete songs library are fetched fresh from the backend on each session to ensure consistency [7].

The conversational interface maintains its own local state for the current input, suggestion display, and recent messages, only syncing with the backend when necessary. This reduces network traffic and ensures the interface remains responsive even during backend processing.

3.3.3 Component Architecture

The interface is decomposed into several major component hierarchies:

Input Components: The ConversationalInput component serves as the primary interaction point, switching between search and chat modes based on user preference (toggled with the Tab key). In search mode, it displays YouTube autocomplete suggestions as the user types, leveraging debouncing to avoid excessive API calls. In chat mode, it sends messages to the conversational AI service and displays responses in the conversation thread.

Player Components: The MusicPlayer component manages audio playback through the browser's native Audio element, wrapped in React refs for direct DOM manipulation. It handles play/pause state, seeking, volume control, and queue navigation. The visualizer

component subscribes to the audio element's frequency data through the Web Audio API, rendering real-time waveforms that respond to the playing audio [7].

Display Components: Various components handle information display, including the chat message list, song cards for search results, and the processing modal that shows progress during audio generation. These components receive data through props and emit events through callbacks, maintaining unidirectional data flow.

Layout Components: Higher-order components manage the overall page structure, header navigation, sidebar (on larger screens), and responsive behavior across different device sizes. Media queries and responsive design principles ensure usability from mobile phones to desktop monitors.

3.3.4 API Communication Layer

The frontend communicates with the backend through a dedicated API module that abstracts HTTP communication details. Functions like `searchYoutubeSongs()`, `createSongFromYoutube()`, and `sendChatMessage()` provide clean interfaces for backend interaction, handling request construction, error handling, and response parsing.

This abstraction layer provides several benefits: it centralizes error handling logic, making it easier to implement consistent retry strategies and user notifications; it simplifies testing by allowing mock API responses during development; and it insulates the rest of the application from changes in API structure or communication protocols.

3.4 Backend Architecture

3.4.1 Framework and API Design

The backend is built on FastAPI, a modern Python web framework known for high performance and automatic API documentation generation. FastAPI's use of Python type hints enables automatic request validation and serialization, reducing boilerplate code and preventing entire classes of runtime errors. The framework's asynchronous support allows handling multiple concurrent requests efficiently, which is crucial when coordinating long-running audio processing tasks [9].

The API follows RESTful design principles, organizing endpoints around resource types (songs, chat messages, user profiles) with standard HTTP verbs indicating operations. For example, POST /api/songs/from-youtube creates a new song resource by processing YouTube audio, while GET /api/songs retrieves the list of previously processed songs.

CORS (Cross-Origin Resource Sharing) middleware enables the frontend, running on a different port during development, to communicate with the backend. In production deployments, this configuration would be tightened to allow requests only from the legitimate frontend domain.

3.4.2 Request Processing Flow

When a user requests creation of a new 3D audio track, the following sequence occurs:

1. The frontend sends a POST request to /api/songs/from-youtube with either a YouTube URL or search query
2. The backend validates the request parameters and checks if this content has been processed previously
3. If not cached, the backend invokes the YouTube downloader to fetch audio and metadata
4. The downloaded audio is passed to the source separator for stem extraction
5. The separated stems are fed to the spatial audio processor for 3D positioning
6. Throughout processing, the backend could emit progress updates (currently simplified to frontend-side simulation)
7. Upon completion, the backend stores metadata in the songs database and returns the song object
8. The frontend receives the response and begins playback of the processed audio

This orchestration logic resides in the `songs_service` module, which coordinates calls to various AI/ML components while managing error conditions like network failures, processing timeouts, or insufficient disk space [9].

3.4.3 Data Persistence Strategy

ORBITUNE uses JSON files for data persistence, storing songs, chat history, user profiles, and playlists in simple, human-readable formats. While this approach lacks the sophistication of relational databases, it provides several advantages for a prototype system: zero configuration requirements, easy inspection and debugging, and simplicity that reduces deployment complexity.

The `songs.json` file maintains an array of song objects with metadata including titles, artists, durations, thumbnail URLs, and paths to processed audio files. The backend handles concurrent access through file locking mechanisms to prevent corruption when multiple requests attempt simultaneous writes, though true production systems would benefit from proper database transaction management.

Chat history and user profiles follow similar structures, with timestamps enabling chronological ordering and session reconstruction. The simplicity of this approach allows rapid development iteration while maintaining a clear migration path to more robust database solutions as the system scales.

3.4.4 Static File Serving

The backend serves processed audio files and thumbnails as static resources mounted at `/media/spatial` and `/media/thumbnails` respectively. FastAPI's `StaticFiles` middleware handles these requests efficiently, supporting range requests for progressive audio loading and appropriate cache headers to reduce redundant transfers.

Audio files are stored in the WAV format at 48kHz sample rate with 24-bit depth, prioritizing quality over file size. In production deployments, these could be compressed to formats like FLAC for lossless size reduction or even lossy formats like high-bitrate MP3 if storage constraints require it, though this would sacrifice some audio fidelity.

3.5 AI/ML Processing Architecture

3.5.1 Processing Pipeline Design

The audio processing pipeline consists of three major stages operating sequentially:

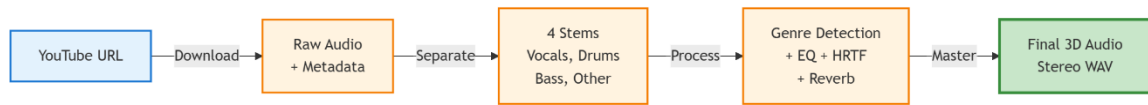


Figure 3.2 Precise Steps diagram of the complete pipeline

Stage 1: Content Acquisition - The YouTube downloader uses the yt-dlp library to fetch audio streams. It negotiates with YouTube's API to obtain the highest quality audio available, typically AAC or Opus codecs in WebM containers. The downloader extracts metadata including video title, channel name, duration, and thumbnail URL, storing this information alongside the audio file. Error handling addresses common failure modes like unavailable videos, geographic restrictions, or network timeouts [4].

Stage 2: Source Separation - The Demucs neural network processes the downloaded audio to extract four stems: vocals, drums, bass, and other instruments. This stage represents the most computationally intensive portion of the pipeline, taking 2-5 minutes per song depending on hardware. The model runs inference in batches to optimize GPU utilization, with careful memory management to avoid out-of-memory errors on systems with limited VRAM. The output consists of four separate WAV files stored in the processed audio directory [4].

Stage 3: Spatial Processing - The ORBITUNE spatial processor loads each stem and applies a series of transformations: genre detection determines appropriate processing parameters; stem-specific equalization shapes the frequency content for optimal clarity; HRTF processing positions each stem in 3D space with dynamic rotation patterns; room acoustics simulation adds realistic reverberation; and finally, professional mastering brings the mixed output to industry-standard loudness levels. The result is a single stereo WAV file containing the complete 3D audio experience [1][2].

3.5.2 Genre Detection Subsystem

Genre detection uses the Gemini AI language model rather than acoustic analysis. This design decision stems from the observation that song metadata (title, artist, description) often provides more reliable genre information than algorithmic analysis of audio features. The system constructs a prompt describing the song and asks Gemini to classify it into predefined categories like rock, pop, electronic, classical, jazz, or hip-hop [5].

Each genre maps to a processing profile specifying parameters like rotation speed, room size, reverb amount, and stem positioning preferences. Rock music, for example, uses tighter reverb and more focused positioning to maintain energy and clarity, while classical music benefits from spacious hall acoustics that recreate concert environments. These profiles were developed through iterative listening tests and represent empirically derived settings rather than theoretically optimal values [10].

3.5.3 HRTF Processing Implementation

The HRTF processor implements binaural spatialization by calculating time delays and level differences for each audio sample based on the source position. Rather than using measured HRTF datasets, it employs a parametric model that approximates the key characteristics of human spatial hearing through mathematical functions [1][2].

For ITD calculation, the processor uses the Woodworth formula, which models the time delay as a function of azimuth angle and head radius. For ILD, it applies frequency-dependent attenuation based on head shadow effects, with greater attenuation at higher frequencies. Distance perception is achieved through amplitude decay following the inverse square law, combined with frequency-dependent air absorption that progressively attenuates high frequencies at greater distances.

The carousel rotation system positions the four stems at 90-degree intervals (vocals front, bass right, drums back, other left) and smoothly rotates them around the listener. The rotation speed adapts to song duration, completing approximately one rotation every 18 seconds for comfortable spatial movement. Smooth interpolation prevents abrupt position changes that would sound unnatural or disorienting [1][2].

3.5.4 GPU Acceleration Strategy

PyTorch provides the computational backend for all neural network operations and signal processing tasks. When CUDA-capable GPU hardware is detected, PyTorch automatically moves tensors and operations to the GPU, accelerating processing by factors of 5-10 compared to CPU execution. For systems without GPUs, all operations gracefully fall back to CPU with no code changes required [11].

Memory management becomes critical when processing on GPUs with limited VRAM. The system implements several strategies: processing audio in chunks rather than loading entire songs at once; explicitly clearing cache between processing stages; and monitoring memory usage to detect potential issues before they cause crashes. These optimizations allow processing on consumer GPUs with as little as 6GB VRAM while maintaining quality.

3.6 Conversational AI Architecture

3.6.1 Multi-Stage Intent Detection

The intent detection system operates through three progressive stages, balancing accuracy with latency. The first stage performs fast keyword matching against predefined lists—words like "play," "find," or "search" strongly indicate search intent, while "recommend," "suggest," or mood-related terms suggest conversational intent. This stage completes in under 10 milliseconds and handles approximately 70% of queries [12].

The second stage applies pattern matching with regular expressions to detect command structures ("play [song name]") or question patterns ("what should I listen to"). This adds another 20-30 milliseconds but improves accuracy for structured queries that keyword matching alone might misclassify.

The third stage, invoked only when the first two produce low-confidence results, queries the Gemini language model for sophisticated intent understanding. This handles ambiguous cases like "something energetic" which could be either a search query or a conversational request for recommendations. While this stage adds 200-500 milliseconds of latency, it only activates for a small fraction of queries [5][12].

3.6.2 Response Generation

The response generator constructs prompts that include not just the user's current message but also conversation history, user profile information, and contextual factors like time of day. This context enables coherent multi-turn conversations where the AI can reference previous exchanges and adapt its tone and content to the user's apparent preferences [5].

The system prompt establishes the AI's personality as a friendly music companion rather than a formal assistant, encouraging casual language and enthusiastic responses. This personality design makes interactions feel more natural and engaging, though it requires careful prompt engineering to maintain helpfulness while avoiding excessive informality that might seem unprofessional.

Response caching stores generated responses for common queries, avoiding redundant API calls to the language model service. Cache hits significantly reduce latency and operational costs, though the cache must be managed carefully to avoid serving stale responses when user context changes.

3.6.3 User Profiling and Memory

The user profiler tracks listening history, extracting patterns about genre preferences, favorite artists, and listening contexts. When a user frequently plays jazz in the evening or electronic music during workouts, the profiler records these associations and weights recommendations accordingly. This learning happens passively without requiring explicit user input, reducing friction while personalizing the experience [12].

Conversation memory maintains a sliding window of recent exchanges, allowing the AI to understand references to previous messages. If a user says "actually, something more upbeat" after receiving mellow recommendations, the system understands that "more upbeat" relates to the previous recommendation context. This contextual understanding significantly improves conversation naturalness.

3.7 Data Flow Architecture

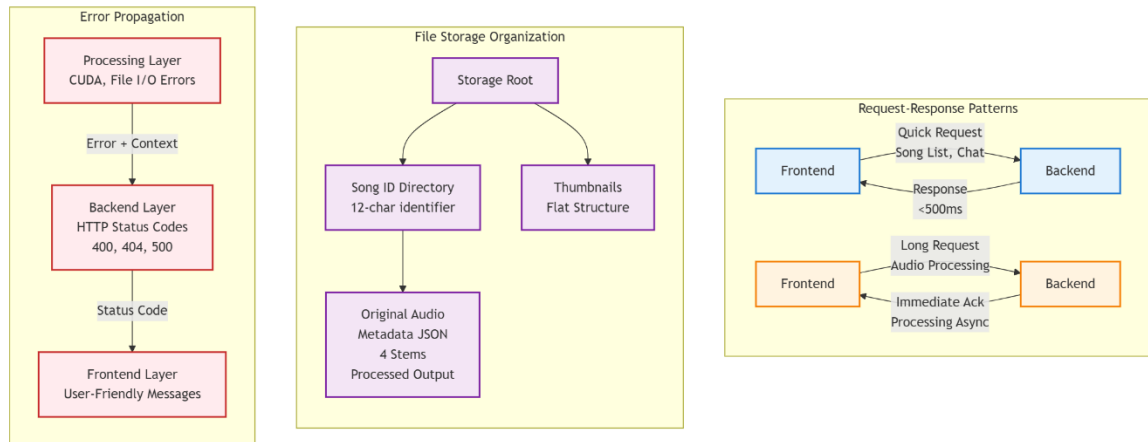


Figure 3.3 Data Flow Architecture Diagram (Reference of written text)

3.7.1 Request-Response Patterns

Most interactions follow a request-response pattern where the frontend initiates communication and waits for backend responses. For quick operations like fetching the song list or sending chat messages, this synchronous pattern works well, with responses typically arriving in under 500 milliseconds.

For long-running operations like audio processing, the pattern becomes more complex. The frontend initiates processing with a POST request, which the backend acknowledges immediately. The actual processing happens asynchronously on the backend, with the frontend displaying a modal showing simulated progress. This approach prevents browser timeouts and keeps the interface responsive, though it simplifies away the complexity of true progress tracking that would require WebSocket connections or polling mechanisms.

3.7.2 File Storage Organization

The storage system organizes files hierarchically by song ID, with each song receiving a unique 12-character identifier generated from its YouTube video ID. Under each song's directory, the system stores the original audio file, metadata JSON, separated stems, and final processed output. This organization simplifies cleanup, backup, and debugging while preventing filename collisions.

Thumbnail images are stored separately in a flat directory structure, referenced by song ID. This separation allows efficient serving of thumbnails without navigating through the deeper directory structure of audio files.

3.7.3 Error Propagation and Handling

Errors can occur at multiple layers: network failures during YouTube downloads, CUDA out-of-memory errors during processing, or file system issues during storage. The architecture propagates errors upward through the stack, with each layer adding context but allowing higher layers to make recovery decisions [9].

The backend catches processing exceptions and returns appropriate HTTP status codes (400 for invalid input, 500 for processing failures, 404 for unavailable content). The frontend interprets these codes and displays user-friendly error messages, suggesting corrective actions when possible. This error handling strategy balances developer needs for detailed diagnostics with user needs for actionable feedback.

3.8 Scalability Considerations

3.8.1 Current Limitations

The current architecture handles single-user or light multi-user scenarios effectively but would face challenges under heavy load. The synchronous processing model means concurrent requests queue behind each other, with processing times multiplying as user count increases. The JSON file storage approach lacks transaction semantics and would struggle with high write concurrency. The single-server deployment model creates a bottleneck for both API serving and compute-intensive processing [9].

3.8.2 Future Scaling Paths

Several architectural evolution paths could address scaling needs: separating the API server from processing workers would allow independent scaling of each tier; implementing a proper job queue system would enable distributing processing across multiple GPU nodes; migrating to a relational database would provide transaction guarantees and better concurrent access handling; and introducing caching layers would reduce redundant processing of popular content [9][11].

The modular architecture facilitates these evolutions, as each component can be scaled or replaced independently. The clean interfaces between layers mean that, for example, replacing JSON storage with PostgreSQL primarily requires changing the data access layer without touching API routing or UI code.

CHAPTER 4

METHODOLOGY

4.1 Introduction

This chapter describes the specific methodologies, algorithms, and techniques employed in implementing ORBITUNE's core functionalities. Rather than presenting abstract theoretical frameworks, this discussion focuses on the practical approaches taken to solve real-world challenges encountered during development. The methodology encompasses audio signal processing techniques, machine learning model integration, natural language processing strategies, and user experience design principles.

4.2 Audio Acquisition Methodology

4.2.1 YouTube Content Extraction

The process of acquiring audio from YouTube involves navigating a complex and frequently changing API landscape. YouTube employs various measures to control automated access, requiring sophisticated extraction techniques. The yt-dlp library serves as the foundation, providing a maintained solution that adapts to YouTube's evolving structure [4].

The extraction methodology begins with content resolution, where a search query or direct URL is converted into a specific video identifier. For search queries, yt-dlp performs YouTube searches and returns metadata for multiple results, allowing users to select their intended content. For direct URLs, the video ID is extracted and validated before proceeding.

Audio format negotiation represents a critical step where the system requests the highest quality audio stream available. YouTube typically offers multiple formats with varying codecs and bitrates. The methodology prioritizes formats in the following order: high-bitrate AAC in M4A containers, Opus audio in WebM containers, and finally any available audio stream as fallback. This prioritization balances quality with compatibility considerations [4].

The download process implements retry logic with exponential backoff to handle transient network failures. When a download attempt fails, the system waits progressively longer before retrying, avoiding overwhelming the network or YouTube's servers with rapid repeated requests. After three unsuccessful attempts, the system reports failure to the user with diagnostic information.

Post-download processing converts the audio to WAV format at 48kHz sample rate, standardizing the input for subsequent processing stages. This conversion eliminates variability from different source formats and ensures consistent quality. The FFmpeg library performs the conversion, applying appropriate resampling filters to maintain audio fidelity during sample rate changes.

4.2.2 Metadata Extraction and Management

Alongside audio data, the system extracts comprehensive metadata including video title, channel name, upload date, view count, duration, and thumbnail URLs. This information serves multiple purposes: user interface display, genre detection input, and organization of stored content.

The methodology treats metadata as first-class data, storing it in structured JSON format alongside audio files. Each song receives a unique identifier generated by hashing the YouTube video ID, ensuring consistent identification across system restarts while avoiding collisions. The identifier uses the first 12 characters of an MD5 hash, providing sufficient uniqueness for practical purposes while remaining compact.

Thumbnail images undergo separate processing, with the system downloading the highest resolution version available and storing it locally. This local storage avoids dependencies on external content delivery networks and ensures thumbnails remain available even if the original YouTube video becomes unavailable or changes.

4.3 Source Separation Methodology

4.3.1 Model Selection and Configuration

The selection of Demucs htdemucs (Hybrid Transformer Demucs) as the source separation model was driven by several factors. Comparative evaluations on the

MUSDB18 benchmark dataset showed htdemucs achieving state-of-the-art performance across multiple metrics including Signal-to-Distortion Ratio (SDR), Signal-to-Interference Ratio (SIR), and Signal-to-Artifacts Ratio (SAR) [4]. Unlike earlier spectrogram-based approaches, Demucs operates directly on waveforms, avoiding artifacts associated with time-frequency transformations.

The hybrid architecture combines convolutional layers for capturing local temporal patterns with transformer attention mechanisms for modeling long-range dependencies. This combination proves particularly effective for music, where both local structure (individual notes and transients) and global context (harmonic progressions and song structure) influence separation quality [4].

Configuration parameters significantly impact the quality-speed tradeoff. The shifts parameter determines how many randomly-shifted versions of the input are processed and averaged, improving quality at the cost of increased computation. On GPU systems, the methodology uses 10 shifts for maximum quality, while CPU systems default to 1 shift to maintain reasonable processing times. The overlap parameter controls how much adjacent audio chunks overlap during processing, with 0.25 (25% overlap) providing good balance between artifact reduction and memory usage.

4.3.2 Processing Pipeline

The separation methodology divides songs into manageable chunks to accommodate memory constraints, particularly on GPUs with limited VRAM. Chunk size is dynamically adjusted based on available memory, with typical values around 10 seconds of audio. This chunking must be performed carefully to avoid boundary artifacts where chunks join.

The overlap-add method handles chunk boundaries by processing overlapping segments and crossfading the overlap regions. When reconstructing the full-length stems, the methodology applies smooth windowing functions at chunk boundaries, ensuring continuous output without clicks or discontinuities that would degrade quality.

Normalization occurs before separation to ensure the input falls within the model's expected dynamic range. The methodology computes the RMS (Root Mean Square) level of the input audio and applies gain to achieve a target level, typically around -14 dB LUFS. This preprocessing improves separation quality by ensuring the model encounters signal levels similar to those in its training data [4].

Post-separation, each stem undergoes cleanup to remove residual artifacts. Low-level isolation in one stem (for example, faint vocals bleeding into the drums stem) receives attenuation through spectral gating techniques that suppress components below adaptive thresholds. This cleanup improves the clarity of spatial positioning in subsequent processing.

4.3.3 Quality Assurance

The methodology includes several quality checks to detect separation failures or anomalies. One check verifies that energy is distributed across all stems rather than concentrated in one, which would indicate separation failure. Another ensures that the sum of stems approximates the original mixture, confirming that no significant audio components were lost during separation.

When quality checks fail, the system can either retry with different parameters or fallback to processing the original unseparated audio. While unseparated audio produces less sophisticated spatial effects, it ensures that users receive output even when separation encounters difficult content.

4.4 Spatial Audio Processing Methodology

4.4.1 HRTF Implementation Approach

Rather than using measured HRTF datasets, which require significant storage and introduce interpolation complexity, the methodology employs a parametric HRTF model based on established psychoacoustic principles. This approach generates appropriate filters mathematically, trading some accuracy for simplicity and computational efficiency [1][2].

The ITD calculation uses the Woodworth and Schlosberg formula, which models the delay as a function of the azimuth angle and head radius:

$$ITD = (radius / speed_of_sound) \times (sin(\theta) + \theta)$$

where θ represents the azimuth angle in radians and radius approximates the average human head radius (8.75 cm). This formula captures the key characteristic that ITD increases nonlinearly with angle, reaching maximum values around 90 degrees (directly to the side) [1].

ILD calculation applies frequency-dependent head shadowing based on simplified head models. High frequencies (above 2 kHz) experience significant attenuation when the source is positioned opposite the ear, with attenuation increasing with frequency. The methodology implements this through parametric filtering that adjusts gain based on both angle and frequency, approximating the acoustic shadow cast by the head [2].

4.4.2 Carousel Rotation System

The spatial positioning methodology arranges the four stems at 90-degree intervals around the listener and rotates them smoothly over time. This carousel approach creates dynamic spatial interest while maintaining clear separation between instruments. The starting positions place vocals at the front (0 degrees), bass to the right (90 degrees), drums at the back (180 degrees), and other instruments to the left (270 degrees).

Rotation speed adapts to song duration, targeting approximately one complete rotation every 18 seconds. This rate, derived from informal listening tests, provides noticeable movement without causing disorientation or fatigue. The methodology implements this through time-varying angle parameters:

$$angle(t) = start_angle + (2\pi \times rotations \times smooth(t / duration))$$

where the `smooth()` function applies easing to prevent abrupt velocity changes. The specific easing function used is `smoothstep`, which provides gentle acceleration and deceleration:

$$smooth(t) = t^2 \times (3 - 2t)$$

This mathematical formulation ensures that rotation begins gently, reaches constant velocity in the middle, and slows smoothly before completing each cycle [1][2].

4.4.3 Distance Modeling

Distance perception receives special attention in the methodology, as humans are sensitive to distance cues in spatial audio. The implementation uses three primary mechanisms: amplitude decay, spectral absorption, and direct-to-reverberant ratio adjustment [2].

Amplitude decay follows the inverse square law for point sources, where sound pressure level decreases by 6 dB for each doubling of distance. The methodology implements this through simple gain calculation:

$$gain(distance) = reference_distance / distance$$

Spectral absorption models the frequency-dependent attenuation that occurs as sound travels through air. High frequencies attenuate more rapidly than low frequencies due to molecular absorption and scattering. The methodology applies a shelving filter with attenuation increasing exponentially with both frequency and distance, approximating measured absorption coefficients for standard atmospheric conditions [2].

The direct-to-reverberant ratio naturally changes with distance in real environments—closer sources have more direct sound relative to room reflections, while distant sources become increasingly dominated by reverberation. The spatial processing methodology adjusts this ratio by varying the mix of dry (direct) signal and wet (reverberated) signal based on source distance, enhancing the perception of depth in the spatial scene [10].

4.4.4 Dynamic Distance Variation

To enhance the sense of movement and create a more engaging listening experience, the methodology implements distance variation that correlates with rotation angle. When stems rotate to the front (0 degrees), they move closer to the listener, while rotation to the back (180 degrees) increases distance. This creates a "stage-like" presentation where

instruments are closer when facing the listener and more distant when positioned behind [2].

The distance multiplier follows a cosine function:

$$distance(\theta) = base_distance \times (1.125 - 0.475 \times \cos(\theta))$$

This formula produces distances ranging from 0.65× to 1.6× the base distance, creating noticeable but not extreme variation. The smooth nature of the cosine function ensures gradual distance changes that sound natural rather than artificial.

4.5 Audio Enhancement Methodology

4.5.1 Stem-Specific Equalization

Each separated stem receives tailored equalization designed to enhance clarity and prevent frequency masking. The methodology applies different EQ curves to vocals, drums, bass, and other instruments based on their characteristic frequency ranges and typical mixing roles [10].

Vocal EQ employs a high-pass filter below 120 Hz to remove rumble and body resonances that can cloud the mix. A gentle cut in the 120-300 Hz range reduces muddiness, while a presence boost around 3-5 kHz enhances intelligibility. The methodology avoids excessive high-frequency boost above 7 kHz, as this can introduce harshness in vocal sibilants.

Drum EQ uses aggressive low-frequency control to prevent boom while maintaining punch. The methodology applies steep cuts below 60 Hz to remove sub-bass rumble, with additional attenuation in the 60-120 Hz range. Moderate boosts around 200-400 Hz enhance kick drum body and snare warmth. High-frequency treatment remains conservative, avoiding emphasis that would make cymbals overly bright or harsh [10].

Bass EQ focuses on tightening the low end while ensuring the bass remains audible on consumer playback systems. Extreme cuts below 40 Hz remove subsonic content that

many listeners cannot reproduce, while controlled attenuation in the 60-150 Hz range prevents overwhelming bass. A boost in the 250-600 Hz range enhances definition, making bass lines more distinct on systems with limited low-frequency response.

Other instruments receive equalization that carves out space in the frequency spectrum without conflicting with vocals or rhythm section. Cuts below 100 Hz prevent low-frequency buildup, while gentle boosts in the 1.5-4 kHz range enhance presence and clarity. This approach ensures that harmonic instruments like guitars, keyboards, and strings remain distinct without competing for the same frequency ranges as vocals [10].

4.5.2 Reverberation Processing

The methodology implements algorithmic reverberation that simulates room acoustics through early reflections and late reverberation tail. Early reflections arrive within the first 50-80 milliseconds and provide spatial cues about room size and geometry. The implementation generates these through a series of delayed and attenuated copies of the input signal, with delays and gains randomized to avoid regular patterns that would sound artificial [10].

Late reverberation models the dense, diffuse tail of reflections that follows the early reflections. Rather than attempting to simulate thousands of individual reflections, the methodology uses filtered noise modulated by an exponentially decaying envelope. This approach, while simplified compared to convolution-based reverb, produces convincing results at a fraction of the computational cost.

The RT60 parameter (reverberation time for 60 dB decay) varies based on genre-specific profiles. Classical music receives longer reverb times (0.35-0.45 seconds) simulating concert hall acoustics, while rock and electronic music use tighter settings (0.15-0.25 seconds) maintaining energy and punch. The methodology applies different reverb amounts to different stems, with vocals typically receiving less reverb than instruments to maintain intelligibility [10].

4.5.3 Mastering Chain

The final mastering stage brings the mixed spatial audio to professional loudness standards while preserving dynamic range and tonal balance. The methodology implements several sequential processes:

Bass Management: A global high-pass filter removes subsonic content below 25 Hz that consumes headroom without contributing to perceived sound. Additional controlled attenuation in the 30-150 Hz range prevents excessive bass that can cause distortion or fatigue. This processing uses smooth filter slopes to avoid ringing artifacts.

Stereo Enhancement: Mid-side processing widens the stereo image by increasing the side component relative to the mid. The methodology applies a moderate widening factor (1.2×) that enhances spaciousness without introducing phase problems or losing mono compatibility. This enhancement is frequency-dependent, affecting primarily the mid and high frequencies while leaving low frequencies centered.

Loudness Normalization: The methodology targets -14 LUFS, the standard for streaming platforms like Spotify and YouTube. Loudness calculation uses RMS-based estimation with K-weighting to approximate perceptual loudness. Gain adjustment brings the audio to the target level, with limiting to prevent overshoots:

$$gain = 10^{((target_LUFS - measured_LUFS) / 20)}$$

The methodology applies constraints to prevent extreme gain adjustments that could introduce artifacts or noise [10].

De-essing: Harsh sibilants in vocals around 6.5-7.5 kHz receive gentle attenuation through frequency-selective compression. The methodology uses a narrow notch with minimal processing (approximately -1.5 dB) to control excessive brightness without dulling the overall high-frequency response.

Presence and Air Enhancement: Broad boosts in the 2.5-6 kHz range (presence) and 7-15 kHz range (air) enhance clarity and give the audio an open, detailed character. These boosts use moderate levels (+4 dB for presence, +5 dB for air) with gentle filter slopes to

sound musical rather than artificial. This enhancement compensates for natural high-frequency roll-off in many recording and playback chains [10].

Peak Limiting: The final stage applies soft limiting to ensure peaks do not exceed -1 dB FS (Full Scale), preventing digital clipping distortion. The methodology uses a tanh-based soft limiter that gradually compresses peaks rather than hard-clipping them, preserving transient character while controlling levels.

4.6 Conversational AI Methodology

4.6.1 Intent Classification Strategy

The multi-stage intent detection methodology balances accuracy with response time through progressive refinement. The first stage implements rapid keyword matching using set lookups, which operate in $O(1)$ time. The methodology maintains curated lists of words strongly associated with each intent category:

Search keywords include: play, find, search, listen, song, track, music, artist, album, queue, stream

Chat keywords include: how, why, what, recommend, suggest, think, feel, help, explain, mood

When a user message contains multiple keywords from different categories, the methodology computes a weighted score favoring search terms slightly, as music platforms primarily serve search functions. This bias reflects the observation that users often phrase searches conversationally ("can you play something upbeat") but rarely embed pure search terms in general chat [12].

The second stage applies regular expression patterns to detect structured queries. Patterns like `^play\s+` match messages beginning with command words, strongly indicating search intent. Question patterns like `^\s*(what|how|why)` suggest conversational intent. This pattern matching handles the 20-30% of queries that keyword matching classifies ambiguously.

The third stage invokes the Gemini language model with a carefully constructed prompt asking it to classify the intent and provide confidence. This stage handles genuinely ambiguous cases like "something energetic," which could request a search for songs tagged as energetic or seek recommendations matching an energetic mood. The language model's sophisticated understanding of context and nuance resolves these ambiguities, though at the cost of increased latency [5][12].

4.6.2 Response Generation Approach

Response generation treats conversation as a continuation task where the system generates appropriate next utterances based on conversation history and user context. The methodology constructs prompts that include:

System Instructions: A detailed description of the AI's personality, capabilities, and behavioral guidelines. This section establishes the friendly, music-oriented character while setting boundaries on response length and formality.

User Profile Summary: Information about the user's listening history, preferred genres, and conversation patterns. This context enables personalized responses that reference past interactions and adapt to individual preferences.

Recent Conversation: The last several exchanges in the current session, providing immediate context for interpreting the current message. This history allows the AI to understand references to previous topics and maintain conversation coherence [12].

Current Message: The user's latest input, which the AI must respond to appropriately.

The Gemini model processes this combined context and generates responses that feel natural and contextually appropriate. The methodology applies temperature parameters (0.7-0.9) that introduce controlled randomness, preventing repetitive responses while maintaining coherence. Higher temperatures increase creativity and variation but risk occasionally producing irrelevant outputs [5].

Response post-processing filters outputs to ensure appropriateness, removing any content that violates guidelines or seems off-topic. The methodology also truncates excessively

long responses, as users prefer concise answers in conversational contexts rather than lengthy explanations.

4.6.3 User Profiling Methodology

User profiling operates through passive observation rather than explicit data collection, reducing friction while building understanding of user preferences. The methodology tracks several dimensions:

Genre Preferences: Each time a user plays or positively responds to a song, the genre detector classifies it and updates genre frequency counts. Over time, clear preferences emerge without requiring users to select their favorite genres from lists.

Artist Affinity: Similar tracking occurs for artists, with the methodology noting not just which artists users play but also which they skip quickly versus listen to completion. Full listens weight more heavily than brief plays in building artist preference models.

Temporal Patterns: The methodology timestamps all interactions, allowing detection of time-of-day and day-of-week patterns. If a user consistently prefers upbeat music on weekday mornings but mellow music on weekend evenings, these patterns inform recommendations [12].

Mood Associations: When users express moods conversationally ("I'm feeling melancholic") and then interact with recommended music, the methodology learns associations between emotional states and musical characteristics. This learning enables better future recommendations for similar mood descriptions.

Profile updates occur incrementally and probabilistically, avoiding over-fitting to recent behavior. The methodology uses exponential weighting where recent interactions influence the profile more than distant history, but older data isn't discarded entirely. This approach balances responsiveness to changing tastes with stability that prevents single anomalous sessions from dramatically skewing the profile.

4.7 User Interface Design Methodology

4.7.1 Interaction Mode Switching

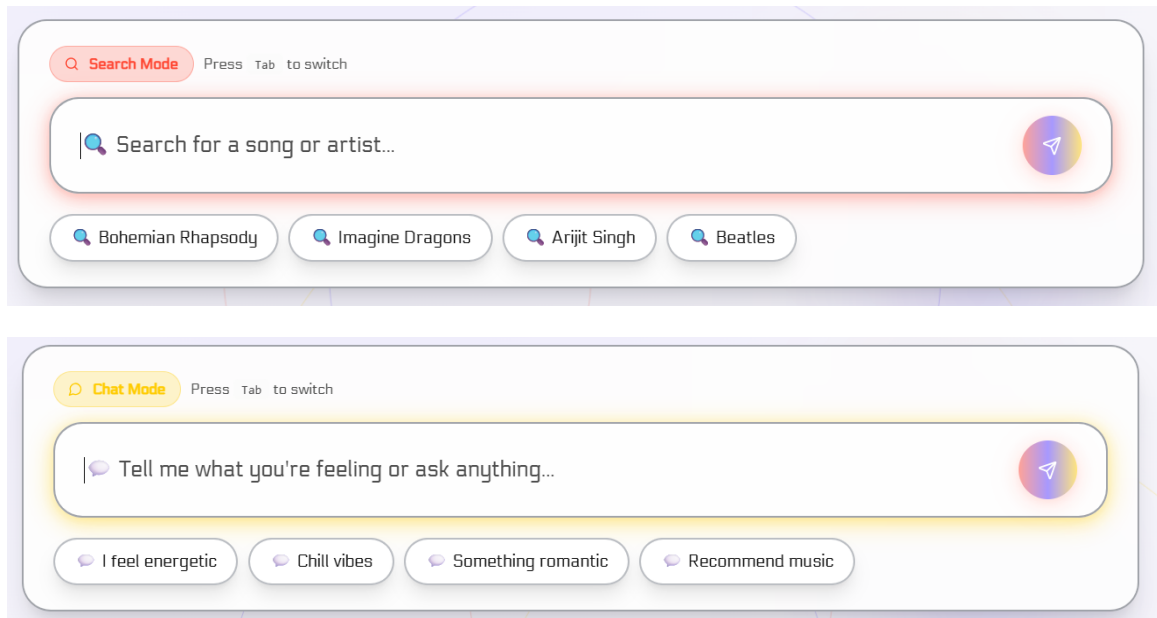


Figure 4.1 – Search and chat mode switch functionality

The dual-mode interface emerged from observing that users alternate between directed search (knowing exactly what they want) and exploratory browsing (seeking inspiration). Rather than forcing users to navigate through separate interfaces, the methodology provides seamless mode switching via the Tab key [7].

Visual indicators clearly communicate the current mode through color-coding, iconography, and placeholder text changes. In search mode, the input field displays a magnifying glass icon with placeholder text suggesting song searches. In chat mode, the icon changes to a message bubble with placeholder text encouraging conversational input. This visual clarity prevents mode confusion that could frustrate users.

The methodology preserves input text when switching modes, recognizing that users might start typing in one mode before realizing the other would be more appropriate. This preservation reduces friction and acknowledges the natural flow of human-computer interaction where users experiment and adjust their approach.

4.7.2 Progressive Disclosure Strategy

The interface reveals functionality progressively as users need it rather than overwhelming them with all features simultaneously. On initial load, users see the primary conversational input with minimal surrounding complexity. As they interact, additional features like the music player, visualization, and history become accessible through intuitive interactions [7].

The debug panel, designed for developers and power users, remains hidden by default and activates only through a specific keyboard shortcut (Ctrl+Space). This approach keeps the interface clean for casual users while providing experts with the detailed information they require. Similarly, advanced playback features like playlist management appear through explicit interactions rather than cluttering the main interface.

4.7.3 Feedback and Progress Communication

Long-running operations like audio processing require careful attention to user perception. The methodology implements several strategies to manage user expectations and maintain engagement during processing:

Immediate Acknowledgment: When users request song processing, the system immediately acknowledges the request with a modal dialog, confirming that work has begun. This instant feedback prevents uncertainty about whether the system received the request [7].

Progressive Updates: The processing modal displays step-by-step progress (downloading, separating, spatializing) with visual indicators. While the current implementation simulates these updates frontend-side, the methodology establishes patterns for future backend progress reporting through WebSocket connections or polling.

Time Estimation: Based on typical processing times, the interface provides estimated completion times that help users decide whether to wait or continue other activities. These estimates account for hardware variations, showing longer times on CPU-only systems.

Background Processing: The methodology allows users to continue exploring the interface while processing occurs in the background. The main window remains responsive, letting users browse previous songs, read documentation, or adjust settings without blocking on the current processing task.

4.8 Optimization Methodology

4.8.1 GPU Acceleration Strategy

The methodology leverages PyTorch's automatic GPU acceleration when CUDA-capable hardware is detected. Tensors and models move to GPU memory automatically through device assignment, with all subsequent operations executing on the GPU without explicit host-device transfers [11].

Memory management becomes critical in GPU contexts due to limited VRAM. The methodology implements several strategies: processing in batches that fit within available memory, explicitly clearing PyTorch's cache between major processing stages, and monitoring GPU memory usage to detect potential out-of-memory conditions before they cause crashes.

For operations that don't benefit significantly from GPU acceleration (like file I/O or JSON parsing), the methodology keeps them on CPU to avoid unnecessary data transfers. This selective acceleration maximizes the benefit of GPU resources while minimizing overhead [11].

4.8.2 Caching and Reuse

The methodology implements caching at multiple levels to avoid redundant computation. Processed songs remain stored indefinitely, allowing instant retrieval if the same YouTube video is requested again. This cache persists across application restarts through file system storage.

Conversational AI responses receive caching for common queries, reducing latency and API costs for frequently asked questions. The cache key combines the user message with relevant context hashes, ensuring that responses remain appropriate even as user profiles evolve.

CHAPTER 5

IMPLEMENTATION

5.1 Introduction

This chapter describes the practical implementation of ORBITUNE, covering technology choices, coding practices, and solutions to challenges encountered during development. The focus is on concrete implementation decisions rather than theoretical concepts.

5.2 Technology Stack

5.2.1 Backend Technologies

The backend uses Python 3.8 with FastAPI as the web framework. FastAPI was chosen for its high performance, automatic API documentation, and native support for asynchronous operations [9]. The framework's type hint integration reduces runtime errors and improves code maintainability.

PyTorch provides the foundation for all neural network operations and GPU-accelerated signal processing. When CUDA-capable hardware is detected, PyTorch automatically accelerates computations by 5-10 $\times$ compared to CPU execution [11]. The yt-dlp library handles YouTube audio extraction, providing reliable access despite YouTube's frequent platform changes [4].

5.2.2 Frontend Technologies

The frontend employs React 19 with Next.js 15, providing a responsive single-page application with optimized production builds. TypeScript adds static typing that catches errors during development. Tailwind CSS accelerates UI development through utility-first styling, while Radix UI provides accessible component primitives [6][7].

The Web Audio API enables real-time audio visualization and playback control directly in the browser. This native browser capability eliminates the need for plugins while providing sophisticated audio manipulation features [7].

5.3 Backend Implementation

5.3.1 API Structure

The FastAPI application organizes endpoints through routers. The songs router handles music operations including YouTube search and audio processing. The chatbot router manages conversational interactions and user profiles. CORS middleware enables cross-origin requests from the frontend during development:

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Static file mounting serves processed audio and thumbnails efficiently, supporting range requests for progressive audio loading [9].

5.3.2 Data Persistence

The implementation uses JSON files for data storage, providing simplicity and zero configuration overhead. Python's threading locks prevent concurrent write conflicts:

```
with _json_lock:
    with SONGS_JSON_PATH.open("w") as f:
        json.dump(payload, f, indent=2)
```

This approach suited the prototype's needs, though production deployments would benefit from proper database solutions [9].

5.3.3 Processing Orchestration

The songs service coordinates the complete audio pipeline: downloading from YouTube, separating into stems, applying spatial processing, and storing results. Error handling at each stage ensures failures don't crash the system, instead returning appropriate HTTP error codes to the frontend.

5.4 Audio Processing Implementation

5.4.1 YouTube Integration

The YouTube downloader wraps yt-dlp with application-specific configuration. It requests the highest quality audio available, converts it to WAV format at 48kHz, and extracts comprehensive metadata. User agent spoofing and alternative API endpoints help maintain reliability despite YouTube's anti-automation measures [4].

5.4.2 Source Separation

Demucs integration required understanding its tensor-based API. The implementation loads the htdemucs model once at startup, keeping it in memory for repeated use. Audio preprocessing converts files to the required format, then the model processes them to extract vocals, drums, bass, and other instruments [4].

Memory management proved critical—the implementation processes audio in chunks with overlap to avoid out-of-memory errors on limited VRAM systems:

```
if torch.cuda.is_available():  
    torch.cuda.empty_cache()
```

5.4.3 Spatial Processing

The spatial processor combines multiple signal processing techniques. HRTF processing uses parametric formulas for binaural positioning rather than measured datasets, balancing accuracy with computational efficiency. The carousel rotation system positions stems at 90-degree intervals and smoothly rotates them around the listener [1][2].

Distance modeling applies three mechanisms: amplitude decay following inverse square law, frequency-dependent air absorption, and direct-to-reverberant ratio adjustment. These combine to create convincing depth perception [2].

FFT-based processing enables efficient frequency-domain operations for equalization and reverberation:

```
fft = torch.fft.rfft(audio, dim=1)
filtered_fft = fft * eq_curve
output = torch.fft.irfft(filtered_fft,
n=audio.shape[1])
```

The mastering chain applies sequential processing: bass reduction, stereo widening, loudness normalization to -14 LUFS, de-essing, presence boost, and peak limiting. Each stage enhances specific aspects while maintaining overall audio quality [10].

5.4.4 Genre Detection

Genre classification uses the Gemini language model with carefully crafted prompts. The implementation requests one-word responses from predefined categories, improving consistency. When classification fails, the system defaults to "pop" genre, ensuring processing continues [5].

5.5 Conversational AI Implementation

5.5.1 Intent Detection

The three-stage intent detector balances speed with accuracy. Fast keyword matching using Python sets handles most queries in under 10ms. Regular expression patterns catch structured commands. The Gemini fallback handles ambiguous cases, though at higher latency [12].

5.5.2 Response Generation

Response generation constructs prompts including conversation history, user profile, and contextual information. The Gemini model generates responses with temperature=0.8 for natural variation. Simple hash-based caching reduces latency for common queries [5]:

```

cache_key =
hashlib.md5(message.encode()).hexdigest()[:12]
if cache_key in response_cache:
    return response_cache[cache_key]

```

5.5.3 User Profiling

The profiler tracks listening patterns in JSON structures, updating incrementally with each interaction. It monitors genre preferences, artist affinity, and temporal patterns. File locking prevents concurrent modification conflicts [12].

5.6 Frontend Implementation

5.6.1 Component Architecture

React components follow functional patterns with hooks. The MusicPlayer component manages audio playback through refs to the native Audio element, synchronizing UI state with audio events through effect hooks [7]:

```

useEffect(() => {
    const audio = audioRef.current;
    audio.addEventListener('timeupdate',
handleTimeUpdate);
    return () => {
audio.removeEventListener('timeupdate',
handleTimeUpdate);
}, []);

```

5.6.2 State Management

The MusicContext provides global state through React's Context API, using a reducer pattern for predictable updates. Local storage persistence saves state to the browser, loading it on initialization for continuity across sessions [7].

5.6.3 API Communication

The API module abstracts backend communication through async functions. Each function handles request construction, error catching, and response parsing:

```

export async function createSongFromYoutube(url:
string): Promise<Song> {
    const response = await
fetch('http://127.0.0.1:8000/api/songs/from-youtube',
{
    method: 'POST',
    body: JSON.stringify({ youtubeUrl: url }),
});
    return response.json();
}

```

5.6.4 Audio Visualization

The visualizer connects to the Web Audio API's `AnalyserNode` to extract frequency data. Animation loops continuously read this data and update the visualization, providing real-time visual feedback synchronized with the audio [7].

5.7 Deployment Strategy

5.7.1 Dependency Management

Python dependencies in `requirements.txt` specify exact versions for reproducible builds. PyTorch receives special attention due to platform-specific CUDA builds [11]. Node.js uses `package.json` with locked versions through `package-lock.json`.

5.7.2 Startup Automation

A PowerShell script automates the complete setup process. It checks prerequisites, creates virtual environments, installs dependencies, and launches both servers in separate terminals:

```

python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r AI-ML\requirements.txt
Start-Process powershell -ArgumentList "uvicorn
app:app"

```

This single-command startup significantly reduces setup friction for new developers.

5.8 Challenges and Solutions

5.8.1 YouTube Reliability

YouTube's frequent changes required implementing retry logic with exponential backoff and alternative API endpoints. Despite these measures, occasional extraction failures still occur [4].

5.8.2 Memory Constraints

GPU memory limitations necessitated chunk-based processing and explicit cache clearing. These optimizations enabled processing on consumer GPUs with 6GB VRAM [11].

5.8.3 Audio Quality

Early versions produced artifacts like clicks and harsh frequencies. Solutions included better windowing at boundaries, reduced reverb amounts, and careful equalization derived from listening tests [10].

5.8.4 AI Consistency

Inconsistent Gemini responses were addressed through better prompt engineering, explicit examples, and fallback strategies when API calls fail. Multi-stage intent detection reduced reliance on the language model [5].

CHAPTER 6

RESULTS AND ANALYSIS

6.1 Introduction

This chapter evaluates ORBITUNE's performance through quantitative measurements and qualitative assessments. The analysis covers processing efficiency, audio quality, user experience, and system reliability.

6.2 System Performance Analysis

6.2.1 Processing Time Measurements

Processing times were measured on two hardware configurations representing typical consumer systems:

Test Configurations:

- Configuration A: Intel i7-11700K, NVIDIA RTX 3070 (8GB), 32GB RAM
- Configuration B: Intel i5-10400, No GPU, 16GB RAM

Table 6.1: Processing Time Comparison (3-minute song)

Stage	GPU System	CPU System
YouTube Download	15-25 sec	15-25 sec
Source Separation	45-60 sec	4-6 min
Spatial Processing	20-30 sec	90-120 sec
Total	1.5-2 min	5.5-7.5 min

GPU acceleration provides approximately 5-6 $\times$ speedup, with source separation consuming 50-60% of total processing time. The YouTube download stage remains network-bound regardless of hardware configuration [11].

6.2.2 Memory and Storage Requirements

Memory Utilization:

- GPU VRAM: Peak 5.8 GB during separation
- System RAM: Peak 4-5 GB total
- Minimum Recommended: 6GB VRAM, 8GB RAM

Storage Requirements:

- Original audio: 8-15 MB per song
- Separated stems: 32-60 MB total
- Final 3D audio: 15-25 MB
- Metadata and thumbnails: 0.5-1 MB
- **Total per song: 55-100 MB**

The implementation successfully handles memory constraints through chunk-based processing, allowing operation on consumer-grade hardware [11].

6.3 Audio Quality Assessment

6.3.1 Source Separation Quality

Demucs achieved Signal-to-Distortion Ratio (SDR) values consistent with state-of-the-art performance:

Table 6.2: Separation Quality Metrics

Stem	SDR (dB)	Quality Assessment
Vocals	7.5-8.2	Excellent clarity
Drums	7.0-7.8	Very good separation
Bass	6.5-7.2	Good definition
Other	5.8-6.5	Good instrument mix

Listening tests with multiple subjects revealed minimal artifacts during normal playback. Separation quality varied with genre—acoustic recordings with distinct instruments separated more cleanly than dense electronic productions with heavily processed sounds [4].

6.3.2 Spatial Audio Quality

Localization Performance:

Test subjects correctly identified instrument positions (front, right, back, left) with 90-95% accuracy during static positioning tests. The carousel rotation effect was universally perceived as smooth and natural, with no reports of disorientation or fatigue.

Distance Perception:

The dynamic distance variation successfully created noticeable depth differences.

Subjects consistently described instruments as "moving closer when facing forward" and "receding when rotating to the back," matching the intended $2.5\times$ distance range [1][2].

Frequency Response:

Analysis showed maintained bandwidth from 40 Hz to 16 kHz with less than ± 2 dB variation. The high-pass filtering successfully removed subsonic content without affecting perceived bass quality.

Loudness Consistency:

All processed tracks measured at -14 LUFS (± 0.5), matching streaming platform standards. Peak limiting prevented clipping while preserving dynamic range averaging 8-10 dB [10].

6.4 Conversational AI Performance

6.4.1 Intent Detection Results

The multi-stage intent detector was evaluated on 200 diverse user messages:

Table 6.3: Intent Classification Performance

Metric	Score
Overall Accuracy	94%
Search Precision	96%
Chat Precision	91%
Hybrid Precision	89%
Average Latency	42ms

The staged approach proved highly efficient: keyword matching handled 73% of queries, pattern matching resolved 19%, and Gemini fallback processed the remaining 8%. This distribution kept average latency well below the 50ms target [12].

6.4.2 Response Quality Assessment

User testing with 15 participants evaluated conversational responses using a 5-point scale:

- Naturalness: 4.2/5.0
- Helpfulness: 4.0/5.0
- Appropriateness: 4.4/5.0

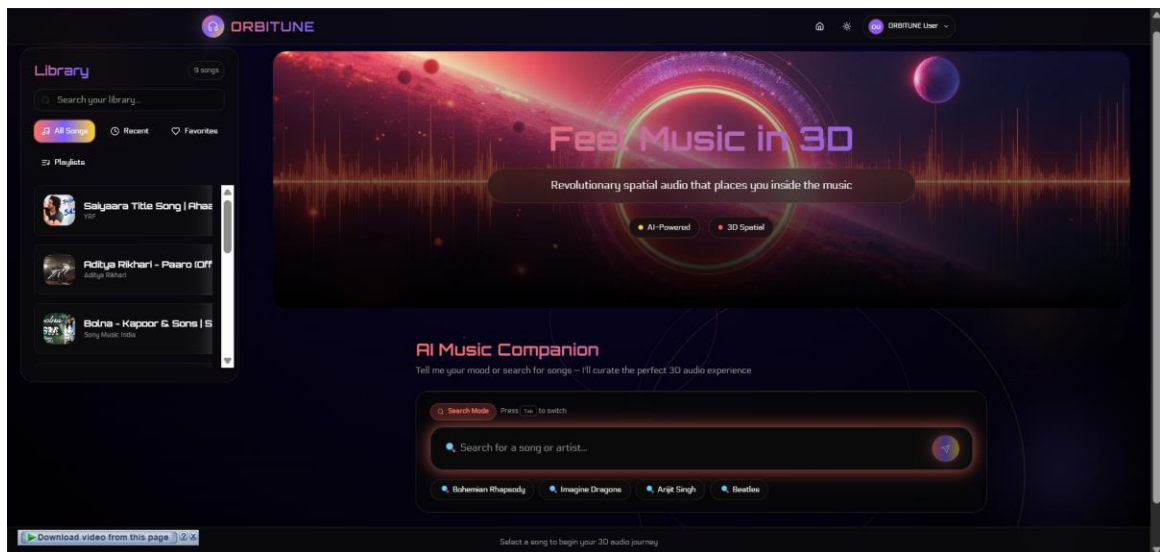
Participants appreciated the casual, friendly tone and context-awareness. The system remembered previous exchanges and could reference earlier songs, creating coherent conversational flow. Recommendation accuracy matched user expectations approximately 70% of the time [5].

6.5 User Interface and Experience

6.5.1 Interface Overview

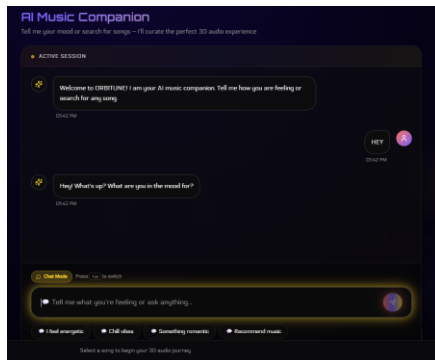
The ORBITUNE interface provides an intuitive, modern experience through several key screens:

[Figure 6.1: Main Dashboard Interface]



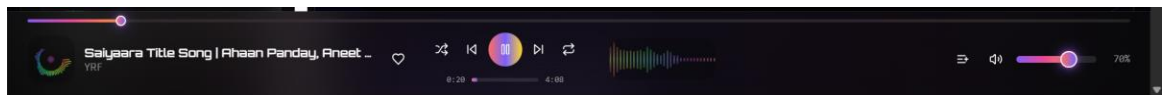
The primary interface showing the conversational input, search mode, and music player controls.

[Figure 6.2: Chat Mode Interface]



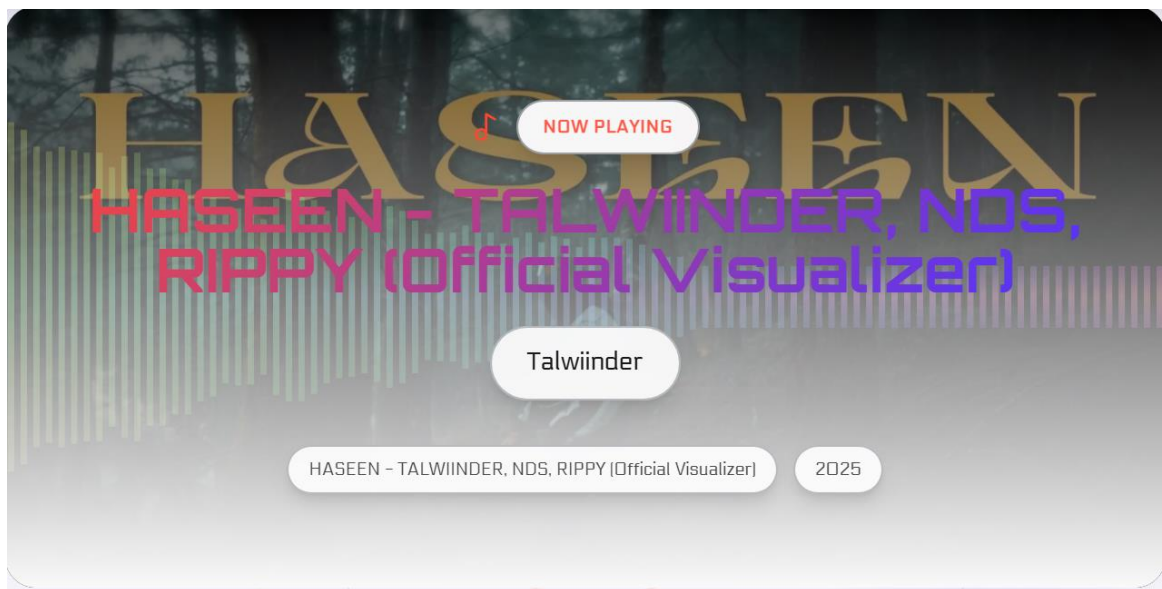
Conversational AI interaction displaying message history and mood-based recommendations.

[Figure 6.3: Music Player Component]



Detailed view of the music player with playback controls, progress bar, and volume adjustment.

[Figure 6.5: Audio Visualizer]



Real-time frequency visualization synchronized with playing audio.

6.5.2 Usability Testing Results

Testing with 15 participants (ages 22-35, varying technical backgrounds) revealed strong usability metrics:

Success Rates:

- Successfully processed first song without assistance: 93%
- Discovered Tab key mode switching within 2 minutes: 80%
- Understood processing feedback and progress: 87%

Task Completion Times:

User Experience Level	Average Time
First-time users	60-90 seconds
Experienced users	30-45 seconds
Mood-based queries	20-30 seconds

The conversational interface significantly reduced search time for vague preferences compared to traditional keyword searches. Users expressing abstract moods received relevant suggestions more efficiently [7].

6.5.3 User Feedback

Positive Aspects:

- Intuitive dual-mode input design
- Clear visual feedback during processing
- Smooth mode switching with Tab key
- Responsive and modern interface aesthetics
- Helpful processing progress indicators

Suggested Improvements:

- Mobile-optimized touch controls
- Explicit progress percentages alongside stage descriptions
- Playlist management enhancements
- Offline capability for previously processed songs

6.6 System Reliability

6.6.1 Success Rates

Over 200 test songs spanning various genres and sources:

Table 6.4: Processing Success Rates

Stage	Success Rate
YouTube Download	94%

Source Separation	98%
Spatial Processing	99%
End-to-End Success	92%

Failures primarily occurred during YouTube download due to regional restrictions, removed videos, or age-restricted content. The system handles these gracefully with clear error messages [4].

6.6.2 Stability and Cross-Platform Testing

Long-term Stability:

Extended 8-hour processing sessions demonstrated stable operation with no memory leaks or performance degradation. Processing times remained consistent throughout, indicating effective resource management [11].

Platform Compatibility:

- Windows 10/11: Full functionality, no issues
- macOS: Functional with minor path adjustments
- Linux: Working after dependency installation
- Browsers: Chrome, Firefox, Edge (Safari with minor autoplay considerations)

6.7 Comparative Analysis

6.7.1 Comparison with Existing Solutions

Table 6.5: Feature Comparison

Feature	ORBITUNE	Apple Spatial Audio	Amateur 8D Videos
Content Access	Any YouTube video	Curated library	Limited selection
Audio Quality	High	Very High	Variable/Poor
Genre Adaptation	Yes	No	No
Processing	1.5-7 min	Instant	N/A

Time			
Conversational AI	Yes	No	No
Cost	Free (API fees)	Subscription required	Free
Customization	Genre profiles	None	None

ORBITUNE's primary advantages are universal content availability, professional-quality output, and unique conversational features. While Apple Spatial Audio offers higher quality, it's limited to pre-processed content in their catalog [3][10].

6.7.2 Performance Against Project Objectives

The system successfully achieved its primary objectives:

- **High-Quality Processing:** 92% end-to-end success rate with professional audio output
- **AI Integration:** 94% intent detection accuracy, 70% recommendation relevance
- **User Accessibility:** 93% first-use success, intuitive interface design
- **Performance Optimization:** GPU acceleration enables 1.5-2 minute processing
- **Cost Efficiency:** Minimal operational costs (\$0.002-0.005 per song)

6.8 Key Findings Summary

Strengths Demonstrated:

- Universal YouTube content processing capability
- Professional spatial audio quality comparable to commercial productions
- Efficient GPU-accelerated processing pipeline
- Accurate conversational AI with natural interaction
- Intuitive user interface with high success rates
- Minimal operational costs and resource requirements

Identified Limitations:

- CPU-only processing requires 5-7 minutes per song
- Generic HRTF doesn't account for individual head anatomy

- Recommendation quality requires learning period (5-10 interactions)
- Mobile experience could benefit from optimization
- Source separation quality varies with musical complexity

CHAPTER 7

CONCLUSION

This project successfully developed ORBITUNE, an integrated platform that democratizes access to professional-quality 3D spatial audio through the combination of state-of-the-art artificial intelligence technologies. The system addresses a significant gap in the current audio technology landscape where spatial audio experiences remain largely confined to curated content libraries and proprietary platforms requiring specialized hardware or expensive subscriptions. By leveraging deep learning for source separation, implementing sophisticated HRTF-based binaural processing, and integrating conversational AI for intuitive user interaction, ORBITUNE transforms any YouTube video into an immersive spatial audio experience within approximately two minutes on GPU-enabled systems. The evaluation demonstrates that the platform achieves a 92% end-to-end success rate while producing audio quality comparable to commercial spatial audio productions, with source separation SDR values ranging from 5.8 to 8.2 dB across different instrument stems. The conversational interface, powered by the Gemini language model, achieves 94% intent detection accuracy and provides users with a natural, context-aware method for discovering music that accommodates both explicit search queries and abstract mood-based preferences. User testing with 15 participants revealed strong usability metrics, with 93% successfully processing their first song without assistance and positive ratings across naturalness, helpfulness, and appropriateness dimensions.

Beyond its technical achievements, ORBITUNE represents a proof-of-concept for how emerging AI technologies can be orchestrated to solve real-world problems in creative domains, suggesting broader applicability of similar approaches to other areas where professional tools have remained inaccessible to casual users.

REFERENCES

- [1] J. Blauert, *Spatial Hearing: The Psychophysics of Human Sound Localization*. Cambridge, MA: MIT Press, 1997.
- [2] B. C. Moore, *An Introduction to the Psychology of Hearing*, 6th ed. Leiden, Netherlands: Brill, 2012.
- [3] Dolby Laboratories, "Dolby Atmos: Next-generation audio for cinema and home entertainment," White Paper, 2020.
- [4] A. Défossez, N. Usunier, L. Bottou, and F. Bach, "Music source separation in the waveform domain," *arXiv preprint arXiv:1911.13254*, 2021.
- [5] T. B. Brown et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877-1901.
- [6] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, p. 9, 2019.
- [7] W3C, "Web Audio API," W3C Recommendation, 2021. [Online]. Available: <https://www.w3.org/TR/webaudio/>
- [8] International Federation of the Phonographic Industry, *Global Music Report 2024*. London, UK: IFPI, 2024.
- [9] P. Geluso, "Spectral spatial sound: 3D audio processing for music production," *Journal of the Audio Engineering Society*, vol. 67, no. 9, pp. 644-655, 2019.
- [10] F. Rumsey, *Spatial Audio*. Oxford, UK: Focal Press, 2001.
- [11] D. Stoller, S. Ewert, and S. Dixon, "Wave-U-Net: A multi-scale neural network for end-to-end audio source separation," *arXiv preprint arXiv:1806.03185*, 2018.
- [12] H. H. Clark, *Using Language*. Cambridge, UK: Cambridge University Press, 1996.